



Instituto Politécnico Nacional
Unidad Profesional Interdisciplinaria en
Ingeniería y Tecnologías Avanzadas

Diseño de propuesta de fragmentación para Ejercicio 3

Alumnos: González Vázquez Rodrigo

Mendoza Xicohtencatl Héctor Jair

Profesor: De la Cruz Sosa Carlos

Materia: Base de datos distribuidas

Grupo: 3TM2

Fecha: 24 de septiembre de 2026

INDICE

Planteamiento	3
Grafo	4
Predicados simples	5
Regla de completitud	8
Reglas de fragmentación	9

Planteamiento

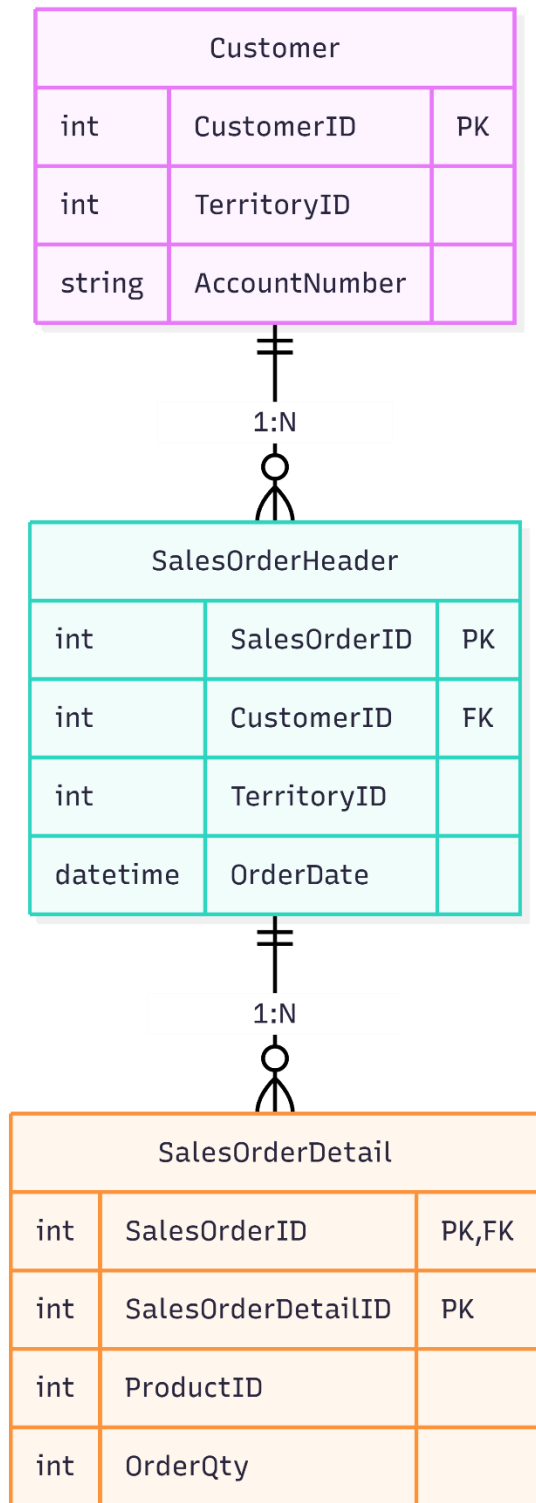
Para iniciar nuestro diseño de fragmentación, en lugar de utilizar un atributo secuencial como **CustomerID**, decidimos trabajar con **TerritoryID** porque agrupar a los clientes por su ubicación geográfica tiene más sentido para la lógica del negocio que agruparlos por su número de registro.

Antes de definir los fragmentos, necesitábamos saber cómo estaban distribuidos los datos. Realizamos una consulta exploratoria sobre el estado actual de la base de datos (que cuenta con más de 19,800 clientes).

```
SELECT
Sales.SalesTerritory.TerritoryID,
Sales.SalesTerritory.Name,
Sales.SalesTerritory.[Group],
COUNT(Sales.Customer.CustomerID) AS TotalClientes
FROM Sales.SalesTerritory
JOIN Sales.Customer
ON Sales.SalesTerritory.TerritoryID = Sales.Customer.TerritoryID
GROUP BY
Sales.SalesTerritory.TerritoryID,
Sales.SalesTerritory.Name,
Sales.SalesTerritory.[Group]
ORDER BY
COUNT(Sales.Customer.CustomerID) DESC;
```

	TerritoryID	Name	Group	TotalClientes
1	4	Southwest	North America	4696
2	9	Australia	Pacific	3665
3	1	Northwest	North America	3520
4	10	United Kingdom	Europe	1991
5	7	France	Europe	1884
6	8	Germany	Europe	1852
7	6	Canada	North America	1791
8	5	Southeast	North America	176
9	3	Central	North America	132
10	2	Northeast	North America	113

Grafo



Predicados simples

Sea C la relación global Sales.Customer. Definimos el conjunto de predicados simples $Pr = \{p_1, p_2, \dots, p_{10}\}$ sobre el atributo TerritoryID, donde cada p_i divide horizontalmente la relación según:

$$p_i: \text{TerritoryID} = i, \quad i \in \{1, 2, \dots, 10\}$$

$$p_{r1}: \text{TerritoryID} = 1$$

$$p_{r2}: \text{TerritoryID} = 2$$

$$p_{r3}: \text{TerritoryID} = 3$$

$$p_{r4}: \text{TerritoryID} = 4$$

$$p_{r5}: \text{TerritoryID} = 5$$

$$p_{r6}: \text{TerritoryID} = 6$$

$$p_{r7}: \text{TerritoryID} = 7$$

$$p_{r8}: \text{TerritoryID} = 8$$

$$p_{r9}: \text{TerritoryID} = 9$$

$$p_{r10}: \text{TerritoryID} = 10$$

Si fragmentábamos directamente por el nombre de la región, por ejemplo, agrupar todos los territorios bajo North America, Europe y Pacific), el resultado era un desbalance: Norteamérica se quedaba con más de registros que el Pacífico 3,600.

Este desbalance rompería el asegurar una distribución uniforme para no sobrecargar un nodo. Por ello, aplicamos una estrategia de agrupación manual, sumando los totales de cada territorio para crear 5 nodos, con un aproximadamente de 3,900 clientes cada uno

- **(Suroeste):** Territorio 4 = **4,696** clientes.

- **(Pacífico y Centro-Noreste):** Territorios 9, 3, 2

$$(3665 + 132 + 113) = \mathbf{3,910 \text{ clientes.}}$$

- **(Europa Occidental):** Territorios 10, 7

$$(1991 + 1884) = \mathbf{3,875 \text{ clientes.}}$$

- **(Noroeste y Sureste):** Territorios 1, 5

$$(3520 + 176) = \mathbf{3,696 \text{ clientes.}}$$

- **(Alemania y Canadá):** Territorios 8, 6

$$(1852 + 1791) = \mathbf{3,643 \text{ clientes}}$$

COM_MIN

Una vez definidos los 5 grupos balanceados, procedimos a aplicar el algoritmo **COM_MIN** para generar nuestros fragmentos primarios.

El algoritmo COM_MIN depura el conjunto inicial Pr eliminando predicados irrelevantes para generar un conjunto óptimo, completo y mínimo denotado como Pr' . A partir de los elementos de este conjunto Pr' se construyen los minitérminos válidos. El algoritmo evalúa nuestros **predicados simples** (por ejemplo: TerritoryID = 4 o TerritoryID IN (1, 5)) y genera los **predicados minitérmino**.

Estos minitérminos son las condiciones exactas que **restringen** lógicamente qué **filas** de **Sales.Customer** van a cada uno de nuestros 5 nodos, garantizando que los fragmentos sean útiles y uniformes.

Formalmente, dado el conjunto de predicados simples Pr , un **minitérmino** m_i se define como la conjunción de cada predicado simple, considerado en su forma natural o en su forma negada (complemento):

$$m_i = \bigwedge_{\{p_k \in Pr\}} p_k^*, \quad \text{donde } p_k^* = p_k \text{ ó } \neg p_k$$

Dado que ***TerritoryID*** es un atributo de valores mutuamente excluyentes (regla de disyunción), cada minitérmino relevante se simplifica a un único predicado positivo:

$$m_i: TerritoryID = i$$

Por ejemplo, si quiéramos agrupar el ID de territorio 10 con el 7 para formar un nodo, tenemos que definir el predicado minitérmino que contuviera esa combinación, y ahí es donde restringimos la información que usaremos:

Fragmento 1 (Suroeste):

$$F_1 = m_4$$

Fragmento 2 (Pacífico y Centro-Noreste):

$$F_2 = m_9 \vee m_3 \vee m_2$$

Fragmento 3 (Europa Occidental):

$$F_3 = m_{10} \vee m_7$$

Fragmento 4 (Noroeste y Sureste):

$$F_4 = m_1 \vee m_5$$

Fragmento 5 (Alemania y Canadá):

$$F_5 = m_8 \vee m_6$$

Fragmento 6 ($n + 1$ para la Regla de Completitud):

$$F_6: \neg(m_1 \vee m_2 \vee \dots \vee m_{10}) \vee (TerritoryID \text{ IS NULL})$$

```
-- NODO 1: Suroeste
SELECT * FROM Sales.Customer
WHERE TerritoryID = 4;
-- NODO 2: Pacífico y Centro-Noreste
SELECT * FROM Sales.Customer
WHERE TerritoryID IN (9, 3, 2);

-- NODO 3: Europa Occidental
SELECT * FROM Sales.Customer
WHERE TerritoryID IN (10, 7);
```

```
-- NODO 4: Noroeste y Sureste
SELECT * FROM Sales.Customer
WHERE TerritoryID IN (1, 5);

-- NODO 5: Alemania y Canadá

SELECT * FROM Sales.Customer
WHERE TerritoryID IN (8, 6);
```

Regla de completitud

La Regla de Completitud y el Fragmento ***n+1***:

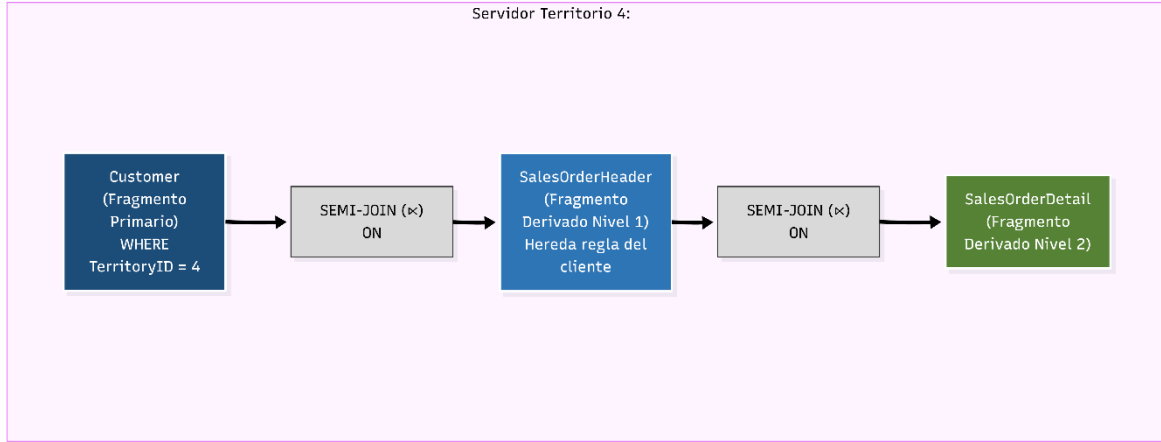
La teoría de Özsu y la regla de completitud dictan que ningún dato puede perderse en la fragmentación. ¿Si mañana la empresa abre un territorio nuevo (ej. TerritoryID = 11) o se inserta un cliente sin territorio asignado (NULL)? Nuestros 5 nodos principales no sabrían qué hacer con ellos.

Para solucionar esto de raíz, el diseño incluye un sexto fragmento, el Fragmento ***n+1***. Este fragmento funciona como una “red de seguridad” que atrapa cualquier territorio futuro o valor nulo

```
SELECT *
FROM Sales.Customer
WHERE TerritoryID NOT IN (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
OR TerritoryID IS NULL
```

Ahora, no solo consiste en clientes. Tenemos relaciones jerárquicas como las órdenes de venta (***SalesOrderHeader***) y detalles (***SalesOrderDetail***), las cuales reciben como llave foránea la llave del cliente (***CustomerID***).

Para mantener la eficiencia y evitar que los datos se separen de sus dueños, diseñamos el **grafo relacional**. Esto significa que aplicamos una **Fragmentación Derivada**.



La fragmentación derivada de **SalesOrderHeader** respecto a su fragmento propietario F_i de Customer se define como:

$$SOH_i = SalesOrderHeader \bowtie_{\{CustomerID\}} F_i$$

De manera análoga, SalesOrderDetail hereda la fragmentación de **SalesOrderHeader**.

$$SOD_i = SalesOrderDetail \bowtie_{\{SalesOrderID\}} SOH_i$$

Donde $(\bowtie)$ denota el operador de semi-join, equivalente en SQL a una cláusula WHERE EXISTS.

La lógica es: Si el Cliente 1 se guarda en el Nodo Norte, todas sus órdenes de compra y los detalles de esas compras heredan esa regla y se van al Nodo Norte.

Para lograr esto sin traer columnas innecesarias o duplicar datos, utilizamos álgebra relacional **SEMI-JOIN** este SEMI-JOIN se traduce a la cláusula EXISTS. Esta técnica conecta la tabla hija con la tabla padre, propagando la fragmentación hacia abajo de manera eficiente

Reglas de fragmentación

Validamos que nuestro diseño final cumpla con las 3 reglas de la fragmentación:

1. **Regla de Completitud:** Se cumple estrictamente gracias a nuestro fragmento adicional $(n+1)$, garantizando que todos los registros (actuales, futuros y atípicos) tengan un nodo asignado.

$\bigcup_{i=1}^6 F_i = \text{Customer}$ — todo registro de la relación global pertenece a algún fragmento.

2. **Regla de Reconstrucción:** Se cumple porque si aplicamos el operador de **UNION** sobre todos nuestros fragmentos (los 5 nodos + el fragmento n+1), podemos reconstruir la tabla global original de **Sales.Customer** sin pérdida de datos.

$\text{Customer} = F_1 \cup F_2 \cup F_3 \cup F_4 \cup F_5 \cup F_6$ mediante el operador de unión.

3. **Regla de Disyunción:** Se cumple porque un cliente solo puede tener un **TerritoryID**, por lo no es posible que un mismo registro resida en dos nodos a la vez.

$F_i \cap F_j = \emptyset, \quad \forall i \neq j$ — ya que TerritoryID toma un único valor por registro.